Lo imprescindible de Python



VARIABLES

Para asignar un valor a una variable:
A = valor

Para consultar el valor de una variable:
A

Para consultar el tipo variable:
type(A)

Para preguntar al usuario el valor de una variable:
A=raw_input("Texto con solicitud")
Por defecto lo toma como string, si queremos otro tipo hay que transformarlo.

Para transformar una variable:
int(A)
float(A)
str(A)

Para publicar texto y variables:
print "El valor de la variable A es " + str(A)

CONDICIONES IF

Para realizar una ejecución condicional, se utiliza if:
if condicion:
sentencias si se cumple
o bien
if condicion:
sentencia si se cumple
else:
sentencia si no se cumple
Para realizar varios if else
if condicion1:
sentencia si se cumple
elif condicion2:
sentencia si se cumple
.
.
.
elif condicion n:
sentencia si se cumple
else:
sentencia si no se cumple ninguna

BUCLE FOR

Permite que una variable vaya cogiendo valores de una lista
for variable in v1,v2,...vn:
sentencias

Podemos tomar los valores de una lista creada antes
lista=[v1,v2,...vn]
for variable in lista:
sentencias

Podemos tomar los n valores de range(n), siendo 0,1,...n-1
lista=[v1,v2,...vn]
for variable in range(n):
sentencias
Posibilidades de range():
range(5)=0,1,2,3,4
range (1,5)= 1,2,3,4 #se inicia en 1
range(2,9,3)=2,5,8 #se inicia en 2 y recorre de 3 en 3

BUCLE WHILE

Permite ejecutar un conjunto de acciones mientras se cumpla una condición
while condicion:
sentencias

Si queremos hacer un bucle infinito:
while True:
sentencias

Si queremos ir aumentando una variable:
i+=1 ó i=i+1. No está permitido i++

Para romper un bucle, aunque no se cumpla la condición de salida, se utiliza el **break** (romper).

Si queremos pasar directamente a la siguiente iteración del bucle, se utiliza **continue** (continuar).

CONDICIÓN Y COMPARACIÓN

and: a and b → se cumple a y b
or: a or b → se cumple a o b
not: not a → No a
==: a==b → a es igual que b
!=: a!=b → a es distinto que b
<: a<b → a es menor que b
>: a>b → a es mayor que b
<=: a<=b → a es menor o igual que b
>=: a>=b → a es mayor o igual que

CADENAS

→ a+b = ab
* → a*3 = aaa
\n → salto de línea
\t → tabulado
\ → caracteres especial→ "\"hola\""= "hola"
[ ] → obtener partes → c="hola"→ c[2] = l . Empieza a contar por 0
len(nombre)→longitud c[1:3] = lo
c[1:]=ola
c[:2]=ho

Las listas y las cadenas son similares, las cadenas pueden ser considerada listas de caracteres que no pueden ser cambiadas.

OPERACIONES ARITMÉTICAS

Suma + → 2+3=5
Negación - → -3
Resta -→ 5-3=2
Multiplicación *→ 2*3=6
Potencias ** → 2**3=8
División / → 8.0/3=2.666
División entera //→ 8.0//3=2.0
Resto %→ 8%3=2

FUNCIONES

Para definir una función:
def nombreFuncion(param1, param2,...,paramn):
sentencias

Para invocar a esa función, simplemente se pone su nombre y se le pasan los parámetros.
Si queremos retornar un valor de una variable utilizaremos:
return variable

MÓDULOS

Son pedazos de código que otras personas han escrito para cumplir tareas comunes tales como generar números aleatorios, realizar operaciones matemáticas, etc

La manera básica de utilizar un módulo es:
```
import nombre_modulo
variable=nombre_modulo.variable(parametros)
```

Ej: random, math...

Así importariamos todos los módulos
```
import random
x=random.randint(1,6)
```

Si queremos importar solo una función o variable:
```
from math import pi
```

LISTA

Son objetos, utilizados para almacenar una lista indexada. Se puede almacenar cualquier tipo de dato y tipos distintos dentro de una misma lista e incluso listas dentro de una lista.

Para asignar un valores a una lista:
```
nombre=[v1, v2, … vn]
```

Para acceder a un valor de la lista:
```
v1 = nombre [0]
v2 = nombre [1]
…
vn = nombre [n-1]
```

Para crear una lista vacía:
```
nombre =  [ ]
```

Para reasignar valores en un determinado índice:
```
nombre[indice] = valor o nombre.insert(indice, valor)
```

Añadir un valor al final de la lista:
```
nombre=nombre+[valor] o nombre.append(valor)
```

Eliminar un valor de la lista:
```
nombre.pop(indice)
```

Numero de elementos de la lista:
```
num=len(nombre)
```

Las listas pueden ser sumadas y multiplicadas como cadenas.
```
nombre = [v1, v2,v3]
nombre = [v4,v5]+nombre  #añada delante
nombre =  nombre *2   #repite 2 veces
```

Obtener el primer índice de un elemento:
```
nombre.index(vn)
```

ABRIR ARCHIVOS

Se pueden leer y escribir los contenidos de archivos. Los de texto son los más fáciles de manipular. Antes de editar un archivo, debe ser abierto.

Para abrir un archivo en modo lectura:
```
archivo=open(“nombrearchivo.txt”) o open(“nombrearchivo.txt”,”r”)
#si el archivo no está en la misma dirección que el archivo habrá que indicar toda la dirección.
```

Para abrir un archivo en modo escritura:
```
archivo=open(“nombrearchivo.txt”,”w”)
```

Una vez abierto y utilizado, debería ser cerrado:
```
archivo.close()
```

Leer todo el contenido del archivo de texto:
```
archivo=open(“nombrearchivo.txt”)
contenido=archivo.read()
```

Leer solo n byte del archivo de texto
```
archivo=open(“nombrearchivo.txt”)
contenido=archivo.read(nbyte)
```

DICCIONARIO

Los diccionarios son estructuras de datos utilizadas para asociar claves a valores. Se puede almacenar cualquier tipo de dato y tipos distintos dentro de un mismo diccionario e incluso listas.

Para asignar un valores a una diccionario:
```
nombre={clave1:valor1, clave2:valor2,... claven:valorn}
```

Para acceder a un valor de la lista:
```
valor1 = nombre [clave1]
valor2 = nombre [clave2]
…
valorn = nombre [claven]
```

Para reasignar valores en un determinada clave:
```
nombre[clave] = valor
```

Añadir una nueva relación clave:valor al diccionario:
```
nombre[clave]=valor     #no debe de existir dicha clave
```

EXCEPCIONES

Excepciones comunes:
- ZeroDivisionError: fallo al dividir entre cero
- ImportError: falla una importación
- IndexError: una lista es indexada con un número fuera de rango
- NameError: una variable desconocida es utilizada
- SyntaxError: el código no puede ser analizado correctamente
- TypeError: función es llamada con un valor de un tipo inapropiado
- ValueError: función es llamada con un valor del tipo correcto pero con un valor incorrecto.

Levantar excepciones, con la sentenciacia try/except:
Si ocurre un excepción, el código del bloque **try** deja de se ser ejecutado y el código en el bloque **except** se ejecuta.

```
try:
   sentencias
except ZeroDivisionError:
   print “Error, no se puede dividir entre 0”
```

Una sentencia except sin ninguna excepción especifica atrapa a todos los errores. Estos deben de utilizarse con moderación ya que pueden atrapar errores inesperados y esconder errores de programación